

Research Summary

“ My research focuses on using advanced AI techniques to automate software testing and program verification, with the vision of automating the production of software that is reliable by construction. ”

Software development traditionally progresses through a structured lifecycle, from requirements and design, to implementation, testing, and maintenance, each stage demanding significant human expertise and coordinated effort. My research focuses on three core stages of this lifecycle, **implementation, testing, and verification**, where software quality can be assured through automated, intelligent methods. In the long term, my research aims to transform software development into an intelligent and trustworthy process in which AI systems understand intent, generate high-quality designs and code, and rigorously validate correctness with minimal human intervention.

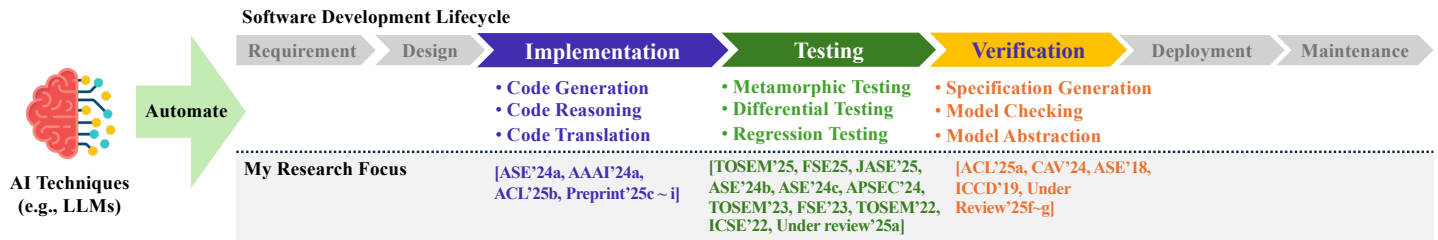


Figure 1 Research Summary

1. AI for Software Implementation

Since the emergence of large language models (LLMs) in 2023, AI techniques have rapidly reshaped software development, with 78% of companies adopting AI in at least one business unit by 2025. A line of my research is building realistic, rigorous foundations for assessing and improving LLMs' coding ability in practical settings.

- **Code Generation** – I developed several benchmarks for realistic code generation. DomainEval [AAAI'25a] exposes LLM weaknesses in domain-specific programming, revealing accuracy gaps of up to 69% across six real-world domains. JavaBench [ASE'25a], built from high-quality undergraduate Java assignments, shows that current models fail to solve even moderate project-level tasks. Moreover, to overcome inherent limitations in LLMs such as knowledge cutoff and hallucination, I conducted extensive studies on retrieval-augmented generation (RAG). My work demonstrates that retrieving external APIs and code examples yields substantial gains, 83% to 220%, especially in fast-evolving library ecosystems. I further show that RAG transfers effectively across 13 programming languages, with Java emerging as a particularly robust bridge for cross-lingual knowledge transfer. Additional analyses identify the key factors driving RAG efficacy, such as the importance of semantically meaningful identifier names and the models' limited ability to generalize from related examples.
- **Code Reasoning** – Beyond generation, I introduced CruxEval-X [ACL'25a], a multilingual benchmark with 19,000+ code reasoning pairs across 19 programming languages. Our findings reveal strong cross-language generalization: despite variability in code generation quality, LLMs exhibit surprisingly consistent reasoning behavior: even when trained only on Python. This resource has been highlighted by reviewers as a valuable foundation for future research in LLM code understanding and model training.

2. AI For Software Testing

Testing is a cornerstone of software quality assurance, yet automating it remains challenging because generating effective inputs, determining correct outputs, and localizing faults all require deep program understanding. My research develops AI-driven methods that automate core testing tasks (e.g., test oracle problem and fault localization) to improve the reliability of both traditional and AI-based software systems.

- **Metamorphic Testing** – The test oracle problem is one of the most fundamental challenges in testing, especially in domains where expected outputs are difficult to obtain. AI systems such as machine translation and chatbots are now ubiquitous, but the oracle problem is further complicated by the scarcity of labeled data, making it hard to determine correct outputs for arbitrary inputs. My research uses metamorphic testing to address this problem. For example, for machine translation systems such as Google Translate, I introduced SemMT [TOSEM'22], a semantic-similarity-based testing framework that reveals errors missed by textual metrics; several issues were later fixed by Google Translate, and the tool is now integrated into the RobustNLP toolkit. For coreference resolution, a preprocess step for natural language processing systems, I proposed a semantic-aware metamorphic testing approach [FSE'23] that synthesizes semantic-equivalent test pairs, reducing false positives from 36% to 0%. For conventional Java projects, I designed MR-Adopt [ASE'24c] to extract implicit relations hidden in developer-written tests to be explicit. By doing

so, the hidden knowledge can be consolidated and further be transferable and generalizable. The experiments show that MR-Adopt can achieve a 72% success rate, which is 33.33% more than baselines.

Furthermore, to support software debugging, I developed DeepFD [ICSE'22], which reframes AI fault localization as a supervised learning task that identifies faulty source-code lines rather than neurons—doubling diagnostic effectiveness and improving accuracy by 82%. I further conducted the first systematic study on using ChatGPT to repair deep-learning programs [JASE'25] and analyzed how prompt design affects automated fixes. This line of work formed my doctoral dissertation, “*Towards Automatic Testing and Fault Localization in Natural Language Processing Systems*” and it was recognized with the **ACM SIGSOFT Outstanding Doctoral Dissertation Award for 2025**, an award presented to only one Ph.D. dissertation in the software engineering area worldwide per year.

- **Differential Testing** – Foundational AI libraries such as TensorFlow, PyTorch, and MXNet form the computational backbone of modern AI systems, but their correctness is difficult to ensure due to complex tensor operations, intertwined control logic, and nondeterminism in training. To detect faults in these libraries, I employed differential testing [TOSEM'23, TOSEM'25], generating inputs that expose inconsistencies across multiple implementations. This approach uncovered 73 previously unknown bugs, all of which were confirmed by developers. By improving the robustness of these core libraries, my work enhances the reliability of the entire AI software stack.
- **Coverage-guided Test Generation** Generating effective test inputs is challenging, especially when targeting hard-to-reach code paths that are only triggered under specific input conditions. Traditional techniques, such as random testing or symbolic execution, often struggle when inputs must satisfy deeply nested or non-linear constraints, like those guarded by complex input validators. To address this, I proposed a hybrid approach that combines the code reasoning capabilities of LLMs with symbolic constraint solvers [ASE'24b]. This method efficiently generates valid inputs that navigate intricate validation logic and significantly enhances bug coverage, achieving 1.4x to 2.3x performance gains over established baselines. This work received the **ACM Distinguished Paper Award** at ASE 2024.

3. AI for Formal Verification

Formal verification provides a rigorous and systematic approach to ensure the correctness and reliability of software systems. Yet, constructing specifications for the full proof relies on domain expertise and non-trivial manpower. Existing automated approaches are often limited in their versatility, i.e., they either focus only on synthesizing loop invariants for numerical programs or are tailored for specific types of programs or invariants. My research builds automated verification pipelines that use AI to synthesize specifications, guide proofs, and construct abstractions, reducing manual effort while improving scalability and practical applicability.

- **Specification Generation from Code:** Given a program and a property to verify, generating sufficient specifications for a complete proof remains challenging, even for LLMs, due to context limitations, iterative error propagation, and occasional hallucinations. To address this, I proposed AutoSpec [CAV'24], which combines static analysis and feedback from a theorem prover. Static analysis breaks the program into manageable pieces, allowing the LLM to focus locally, while the prover validates and retains correct specifications for iterative refinement. AutoSpec fully verified 79% of programs, a 1.59x improvement, and was successfully applied to real-world code, including the X.509 parser project.
- **Specification Generation from Natural Language:** Beyond synthesizing specifications from code, it is also possible to generate them directly from natural language descriptions. As Fields Medalist Terence Tao envisioned, “In the future, instead of typing up our proofs, we would explain them to some GPT.” Inspired by this, I decomposed the pipeline from informal requirements to formal, verifiable proofs into six subtasks. This modular design allows for targeted evaluation of LLMs’ strengths and weaknesses across stages. To support this, I constructed a dataset of 18k high-quality instruction-response pairs across five major formal specification languages (Coq, Lean4, Dafny, ACSL, and TLA+), resulting in the FM-Alpaca (14k+ examples for fine-tuning) and FM-Bench (4k for evaluation) datasets [ACL'25a]. Fine-tuning on FM-Alpaca significantly improved performance not only in formal verification, but also in mathematics, reasoning, and code generation, providing empirical evidence of cross-domain transfer. The work gained strong community interest: 38k+ blog views, 2k+ shares, **2k+ model downloads, and 2k+ dataset downloads**.
- **Model Checking** – Model checking systematically explores a system's state space to verify whether it satisfies specified properties. However, its scalability is limited by the state explosion problem. A common solution is model abstraction, where a simplified model is constructed to preserve the properties of interest, typically through manual effort. To overcome this limitation, I proposed L-CMP [ASE'18 Tool Demo, ICCD'19], a learning-based approach that automates the abstraction process using association rule learning to strengthen the guard of the abstracted predicates. This series of study significantly reduces the state space size and manual overhead, enabling the scalable abstraction and parameterized model checking.

Future Research Plan

My long-term research vision is to establish the scientific foundations for *reliable-by-construction automated software development*. As AI tools become deeply embedded in programming workflows, ensuring that AI-generated artifacts behave correctly, predictably, and in accordance with user intent is a long-term challenge.

1. Short-Term Plan: Formally Modeling Program Behavior

Understanding program behavior is foundational to reliability: AI cannot validate, refine, or reason about code unless it has access to a structured behavioral model. Thus, in the short term, I plan to focus on a concrete motivating question:

“How to obtain precise, machine-checkable models of program behavior from code?”

- **Key research challenges:** (1) Programs often lack explicit specifications; (2) Existing tools (e.g., model checkers) are overly expensive; (3) The generated specifications are often incorrect or incomplete for the final proof.
- **Research directions:** A central objective of this period is to develop testing-guided specification inference. I plan to develop methodologies that infer behavioral models, including formal specifications, state machines, invariants, temporal properties, and path conditions from: partially written code, examples and test cases, execution traces, and natural-language descriptions in code comments if any.
- **Research foundation:** This builds on my prior work in [CAV’24, ACL’25a, ASE’24c, ASE’18, ICCD’19].
- **Expected outcomes:** (1) Tools for automatically extracting behavioral models of AI-generated code; (2) Hybrid static and dynamic analysis pipelines powered by LLM-guided testing; (3) Benchmarks for NL-guided testing and specification inference; (4) Publications in ICSE, FSE, ISSTA, CAV, FM.

2. Mid-Term Plan: Aligning Program Behavior with User Requirements

Bridging the gap between what users mean and what programs do requires integrating behavioral models with iterative refinement, validation, and human-in-the-loop feedback. Thus, in the medium term, I plan to exploring:

“Once we model a program’s behavior, how can we ensure that it actually matches user intent?”

- **Key research challenges:** (1) Requirements are ambiguous and often expressed in natural language; (2) Code at scale may locally satisfy tests but violates global constraints; (3) Misalignment often arises from missing assumptions or hidden preconditions; (4) Validation is expected to scale to multi-module and multi-stage development.
- **Research directions:** A central objective of this period is to ground high-level, informal requirements into structured, testable, and formally checkable semantic representations. By jointly using contrastive tests, property-based tests, and counterexamples-to-correctness, the requirement schemas, scenario-based behavioral expectations, logical predicates, and temporal properties will be inferred with minimal refinement from human inputs. Failures, whether uncovered by tests or proofs, will drive iterative clarification of user intent and correctness-guided repair of code and requirements.
- **Expected outcomes:** (1) Test-driven requirement clarification and disambiguation methodologies; (2) End-to-end workflows that align requirements → tests → specifications → code; (3) Datasets for requirement–behavior alignment and error tracing; (4) Scalable approaches to multi-stage semantic consistency.

3. Long-Term Plan: Foundations for Trustworthy, Reliable-by-Construction AI Development

In the long run, it leads to my long-term career motivating question:

“How to develop foundations for reliable-by-construction automated software development?”

- **Foundational research challenges:** (1) Correctness is expected to be integrated throughout the entire automated AI-driven development process, rather than treated as a post hoc validation step. This requires blending testing, symbolic reasoning, and formal verification into workflows that enforce correctness during synthesis and refinement. In this paradigm, tests and verification become indivisible, they evolve into constructive evidence that informs specification learning, invariant inference, and correctness-preserving code generation. (2) Transforming human intent into structured, testable, and verifiable artifacts involves creating theories and representations that map informal human language to requirements expressed in semantically grounded, formally checkable forms. (3) Advancing trustworthy AI development requires unifying statistical learning with formal reasoning in a principled way. Future AI systems must combine the generalization capabilities of large models with the rigor of symbolic reasoning, enabling hybrid systems that can learn patterns, check themselves, and provide formal guarantees about their outputs.
- **Long-term outcome:** (1) A theory and methodology for reliable-by-design AI development; (2) Architectures that unify testing, verification, and learning; (3) AI systems that autonomously generate verified software at scale; (4) A new paradigm of robust, interpretable, formally grounded AI-driven development.